



THE UNIVERSITY OF CAMBODIA

In Pursuit of Knowledge and Wisdom

PROJECT REPORT

Blockchain Ledger Simulator

An Educational Java Simulator of SHA-256 Hashing,
Block Linking, and Tamper Detection

Course	ITE412 (B)
Lecturer	Hang Youlay
Group	Group 1
Date	September 14, 2026

TEAM MEMBERS

1. Yen Phary
2. Chroeng Simleng
3. Kech Kheang
4. Phort Sopheakdei
5. Samouen Rachana
6. Sroem Meng
7. Phorn Sros
8. Ros Sopheak
9. Seng Tharom

1. Introduction

This project is a small Java program made for learning. It shows the basic ideas behind a blockchain — how data is "locked" using a hash, and how blocks are connected to each other — without the complicated parts of a real cryptocurrency system.

The project focuses on three simple ideas:

- 1 Each block stores some transaction data (example: "Alice pays Bob \$50") plus some extra info about itself.
- 2 Each block creates its own unique code, called a hash, using something called SHA-256.
- 3 Each new block stores the hash of the block before it. This connects all the blocks into a chain. If someone changes an old block, the chain becomes "broken" (invalid).

2. Project Objective

Our goal was to build a simple console program (text-only, no graphics) that can:

- Create the first block, called the genesis block
- Add several transaction blocks after it
- Show the whole chain on the screen
- Check if the chain is still valid (unchanged)
- Show what happens when someone secretly changes a block — the chain should catch it

On purpose, we did NOT include some advanced features that real blockchains have, such as:

- Networking (multiple computers talking to each other)
- Wallets
- Digital signatures
- Mining or proof of work
- Consensus (computers agreeing with each other)
- Databases

These features are important in real systems, but adding them would make it harder to focus on the main lesson: hashing and linking blocks.

3. Blockchain Structure

Our code has four Java files (classes):

- `HashUtil` — creates the SHA-256 hash codes
- `Block` — represents one entry (one block) in the ledger
- `Blockchain` — manages the full list of blocks and checks if the chain is valid
- `Main` — runs the demonstration, used for our presentation/video

Each `Block` stores this information:

Field	What it means
<code>index</code>	The block's position in the chain (0, 1, 2, 3...)

Field	What it means
timestamp	The exact time the block was created
data	The transaction text (example: a fake payment)
previousHash	The hash of the block right before this one
hash	This block's own SHA-256 hash

The very first block is called the genesis block. It always has `index = 0`, `previousHash = "0"` (since there's nothing before it), and `data = "Genesis Block"`. After that, we add more blocks using a method called `addBlock(String data)`.

4. SHA-256 Hashing

To create a hash, we use a tool that is already built into Java, called `MessageDigest`, with a method named "SHA-256". This tool turns any text into a fixed-length code made of letters and numbers.

To create a block's hash, we combine these pieces of information together:

- `index`
- `timestamp`
- `data`
- `previousHash`

An important property of SHA-256 is: if you give it the exact same information, you always get the exact same hash back. But if you change even one small thing (a single letter or number), the hash comes out completely different. This is exactly how we detect if someone has tampered with (secretly changed) a block.

5. Block Linking

Here is what happens, step by step, when we add a new block:

- 1 We look at the most recent block already in the chain
- 2 We give the new block the next index number
- 3 We record the current time as its timestamp
- 4 We copy the previous block's hash into our new block's `previousHash` field
- 5 We calculate the new block's own SHA-256 hash and save it
- 6 We add the finished block to the chain

This creates a one-directional link between blocks, like this:

```
Block #0's hash → stored inside Block #1 as "previousHash"
Block #1's hash → stored inside Block #2 as "previousHash"
Block #2's hash → stored inside Block #3 as "previousHash"
```

Because of this, if someone changes an old block, the newer blocks will no longer correctly "point back" to it — this is how the chain shows something is wrong.

6. Chain Verification (Checking If the Chain Is Valid)

We wrote a method called `getValidationErrors()` that checks every single block and makes a list of anything wrong that it finds. Another method, `isValid()`, simply returns "true" if that list is empty (no problems found).

Here's what the checker looks for:

- 1 For every block (including the genesis block), it recalculates the hash using the block's current data, then compares it to the hash that was originally stored. If they don't match, something changed.
- 2 It checks that the genesis block's `previousHash` is still "0".
- 3 For every block after genesis, it checks that its stored `previousHash` matches the hash we get by *recalculating* the previous block's hash right now (not just trusting the old stored value).

This last point matters a lot for teaching purposes. Because we recalculate instead of just comparing old stored values, when someone tampers with Block #2, our program can catch and clearly report BOTH of these problems:

- Block #2's stored hash no longer matches what we calculate from its (changed) data
- Block #3 is still pointing to Block #2's *original* hash, which no longer exists

7. Tampering Demonstration (Simulating an Attack)

Here is the step-by-step flow used in our Main demonstration:

- 1 Create the blockchain (this automatically includes the genesis block)
- 2 Enter transactions interactively, or press Enter to use five examples
- 3 Print the entire chain, including the exact SHA-256 input and computed hash
- 4 Check that the chain is valid — it should say **VALID**
- 5 Choose a block index and enter replacement data without updating its saved hash
- 6 Check the chain again — this time it should say **INVALID**, and explain exactly why

One important detail: in our code, the methods `Block.setData(...)` and `Block.setPreviousHash(...)` do NOT automatically recalculate the hash when you use them. This is on purpose! In a normal, honest situation, a new hash would always be calculated whenever data changes. By skipping that step here, we are simulating what an unauthorized/sneaky change would look like — someone changing the data but "forgetting" (or being unable) to fix the hash to match.

8. Implementation Details (Tools We Used)

What we used to build this:

- Java 25
- Only Java's built-in standard library — no outside tools or downloads needed
- A simple console (text) interface
- No Maven or Gradle (tools that manage external code libraries) — we didn't need them
- No JUnit (a common testing tool) — instead we wrote our own simple `BlockchainTest` class that checks things step by step

What we focused on for code quality:

- Each class has one clear job
- Methods (functions) are kept short and easy to follow
- We used named constants (fixed values with clear names) for things like the genesis block's info and demo text
- We only added comments where they help explain the blockchain concept, not just to explain basic code

We designed the project so that a student could open `Block.java` and `Blockchain.java` and explain the whole hashing and linking idea in just a few minutes.

9. Testing and Results

Our `BlockchainTest` file checks five situations:

- 1 **Normal chain** — we add five transactions after the genesis block; we expect the result to be `VALID`.
- 2 **Modified block data** — we change the data inside Block #2; we expect the result to be `INVALID`, because the hash won't match anymore.
- 3 **Modified previous hash** — we directly change a block's `previousHash` value; we expect the result to be `INVALID`, because the link between blocks is now broken.
- 4 **Five or more blocks with hashes** — we confirm the chain has the genesis block plus at least five more blocks, and that all of them have valid SHA-256 hashes.
- 5 **Exposed hash input** — we hash the displayed input again and confirm that it produces the block's stored hash.

When we run `Main` by hand, we see the same result: the untouched chain shows as valid, and once we tamper with it, the program shows clear "[INVALID]" messages explaining exactly what went wrong.

9.1 Method of the Live Experiment

To confirm that hashing and validation happen at runtime, we ran:

```
java -cp out Main
```

We entered three original values (100, 150, 200) instead of the sample transactions. After the first validation passed, we selected Block #2 and changed its data from 150 to 900. The stored hash of Block #2 was left unchanged. The full console transcript is reproduced below, then analyzed.

9.2 Full Console Transcript

```
adamreamsmey@Adams-M4-Pro simple-ledger-simulator % java -cp out Main
=== STEP 1: CREATE BLOCKCHAIN ===
Genesis block created automatically.

=== STEP 2: ADD TRANSACTIONS ===
Enter transaction (or 'done' to stop, Enter for samples): 100
Added: 100
Enter transaction (or 'done' to stop, Enter for samples): 150
Added: 150
Enter transaction (or 'done' to stop, Enter for samples): 200
Added: 200
```

Enter transaction (or 'done' to stop, Enter for samples): done

=== STEP 3: DISPLAY BLOCKCHAIN ===

=====

BLOCK #0

=====

Timestamp : 2026-09-14 19:15:51.649

Data : Genesis Block

Previous Hash: 0

Hashing input : "0#1789388151649#Genesis Block#0"

Computed hash : 836e2d8396b4eb0fda3e6d14b66a48ef8e004773ba1f5dcfccebf2ffca9fe66a

=====

=====

BLOCK #1

=====

Timestamp : 2026-09-14 19:15:53.350

Data : 100

Previous Hash: 836e2d8396b4eb0fda3e6d14b66a48ef8e004773ba1f5dcfccebf2ffca9fe66a

Hashing input : "1#1789388153350#100

#836e2d8396b4eb0fda3e6d14b66a48ef8e004773ba1f5dcfccebf2ffca9fe66a"

Computed hash : 5873da25e4af63bae4cda4c30cc81055a715e6b51fc78627a2240eef8997b773

=====

=====

BLOCK #2

=====

Timestamp : 2026-09-14 19:15:54.467

Data : 150

Previous Hash: 5873da25e4af63bae4cda4c30cc81055a715e6b51fc78627a2240eef8997b773

Hashing input : "2#1789388154467#150

#5873da25e4af63bae4cda4c30cc81055a715e6b51fc78627a2240eef8997b773"

Computed hash : ea5e73908cf4e0d366d383ae6c285ef91c214ec240b34d2830a9ceec365f26d8

=====

=====

BLOCK #3

=====

Timestamp : 2026-09-14 19:15:55.205

Data : 200

Previous Hash: ea5e73908cf4e0d366d383ae6c285ef91c214ec240b34d2830a9ceec365f26d8

Hashing input : "3#1789388155205#200

#ea5e73908cf4e0d366d383ae6c285ef91c214ec240b34d2830a9ceec365f26d8"

Computed hash : 4d13f8a7db7b99479186ac75ce2e3a87a4fd73f1283e1d2d45efb3cf6cf3caba

=====

=== VERIFYING BLOCKCHAIN ===

Block #0:

stored=836e2d8396b4eb0fda3e6d14b66a48ef8e004773ba1f5dcfccebf2ffca9fe66a

recalculated=836e2d8396b4eb0fda3e6d14b66a48ef8e004773ba1f5dcfccebf2ffca9fe66a

-> MATCH

Block #1:

stored=5873da25e4af63bae4cda4c30cc81055a715e6b51fc78627a2240eef8997b773

recalculated=5873da25e4af63bae4cda4c30cc81055a715e6b51fc78627a2240eef8997b773

-> MATCH

Block #2:

stored=ea5e73908cf4e0d366d383ae6c285ef91c214ec240b34d2830a9ceec365f26d8

recalculated=ea5e73908cf4e0d366d383ae6c285ef91c214ec240b34d2830a9ceec365f26d8

-> MATCH

Block #3:

stored=4d13f8a7db7b99479186ac75ce2e3a87a4fd73f1283e1d2d45efb3cf6cf3caba

recalculated=4d13f8a7db7b99479186ac75ce2e3a87a4fd73f1283e1d2d45efb3cf6cf3caba

-> MATCH

```
Blockchain is VALID.

=== STEP 5: DEMONSTRATE TAMPERING ===
Enter block index to tamper with: 2
Enter new data for that block: 900
Tampering with Block #2...
Original: 150
Tampered: 900
Warning: stored hash was NOT recalculated.

=== VERIFYING BLOCKCHAIN ===

Block #0:
  stored=836e2d8396b4eb0fda3e6d14b66a48ef8e004773ba1f5dcfccebf2ffca9fe66a
  recalculated=836e2d8396b4eb0fda3e6d14b66a48ef8e004773ba1f5dcfccebf2ffca9fe66a
  -> MATCH
Block #1:
  stored=5873da25e4af63bae4cda4c30cc81055a715e6b51fc78627a2240eef8997b773
  recalculated=5873da25e4af63bae4cda4c30cc81055a715e6b51fc78627a2240eef8997b773
  -> MATCH
Block #2:
  stored=ea5e73908cf4e0d366d383ae6c285ef91c214ec240b34d2830a9ceec365f26d8
  recalculated=7fefc8527f994324139c9362417de1de6d78cd77d4bdcb26a8fbbaca465e0c3
  -> MISMATCH
Block #3:
  stored=4d13f8a7db7b99479186ac75ce2e3a87a4fd73f1283e1d2d45efb3cf6cf3caba
  recalculated=4d13f8a7db7b99479186ac75ce2e3a87a4fd73f1283e1d2d45efb3cf6cf3caba
  -> MATCH

Blockchain is INVALID.

Reason:
[INVALID] Block #2 hash does not match its calculated hash.
[INVALID] Block #3 previousHash does not match Block #2.
```

9.3 Analysis of the Transcript

1. The hashes were computed from live input.

The transaction values 100, 150, and 200 appear inside the hashing-input strings of Blocks #1, #2, and #3. Those values were typed at the console during this run. They are not the program's sample transactions (Alice -> Bob: \$100, and so on). Because SHA-256 is deterministic, a different data field produces a different digest. The hashes in this transcript therefore belong to this experiment, not to a canned example.

2. Each hash is produced from a visible, reconstructable input.

The hashing input uses the form `index#timestamp#data#previousHash`. For Block #2 before tampering, that input was:

```
2#1789388154467#150#5873da25e4af63bae4cda4c30cc81055a715e6b51fc78627a2240eef8997b773
```

The computed hash of that string was:

```
ea5e73908cf4e0d366d383ae6c285ef91c214ec240b34d2830a9ceec365f26d8
```

This makes the hashing step observable. The audience can see exactly which bytes were hashed, rather than treating the digest as a mysterious identifier.

3. The chain is linked by copying each hash into the next block.

Current block	Previous Hash field	Hash of the previous block
Block #1	836e2d83...ca9fe66a	Block #0 hash 836e2d83...ca9fe66a
Block #2	5873da25...8997b773	Block #1 hash 5873da25...8997b773
Block #3	ea5e7390...365f26d8	Block #2 hash ea5e7390...365f26d8

Every previousHash is an exact copy of the previous block's stored hash. Genesis uses "0" because no earlier block exists.

4. Timestamps prove that blocks were created at different times.

The printed times are 19:15:51.649, 19:15:53.350, 19:15:54.467, and 19:15:55.205. The millisecond values differ because the program waits briefly between block creation and because the user typed each transaction by hand. If the output were a static screenshot of hardcoded data, those timestamps would not change from run to run.

5. Before tampering, stored hashes equal freshly recalculated hashes.

The first verification printed MATCH for Blocks #0 through #3. Recalculation uses the same SHA-256 function and the same current contents. Equality means no field used in the hash input had been changed after the block was created. The program therefore reported Blockchain is VALID.

6. Tampering changed Block #2 data but not its stored hash.

The original data of Block #2 was 150. After the user entered 900, the program reported:

```
Original: 150
Tampered: 900
Warning: stored hash was NOT recalculated.
```

This is the experimental treatment. An honest node that created a new block would hash the new contents. Here, setData() updates only the data field. The stored digest remains:

```
ea5e73908cf4e0d366d383ae6c285ef91c214ec240b34d2830a9ceec365f26d8
```

which is still the hash of the old input containing 150.

7. After tampering, two independent checks fail.

The second verification still shows MATCH for Blocks #0, #1, and #3. Those blocks were not modified, so their stored hashes still describe their current contents.

Block #2 shows MISMATCH:

- stored: ea5e73908cf4e0d366d383ae6c285ef91c214ec240b34d2830a9ceec365f26d8
- recalculated: 7fefc8527f994324139c9362417de1de6d78cd77d4bdcbe26a8fbbaca465e0c3

The recalculated value is SHA-256 of the new input that now contains 900. Avalanche effect in SHA-256 explains why the two hex strings share almost no prefix. Changing three characters of data changed the entire digest.

Block #3 still has a MATCH on its own stored hash because Block #3's fields were not edited. Its previousHash, however, is still the old Block #2 hash ea5e73... After tampering, the true hash of Block #2's current contents is 7fefc8... Validation therefore reports a second error:

```
[INVALID] Block #3 previousHash does not match Block #2.
```

This is the educational point of the experiment. Integrity failure on Block #2 (stored hash versus current contents) and linking failure on Block #3 (stale pointer to the previous block) appear together from a single unauthorized edit.

8. What the experiment does not claim.

This run does not prove resistance to a determined attacker who recalculates every later hash, nor does it model mining, consensus, or a network of nodes. It only shows that, in this ledger, an in-place data change that leaves stored hashes untouched is detectable by comparing stored digests with freshly computed ones and by checking previousHash against the previous block's current hash.

10. Conclusion

This project helps show why blockchain-style systems are good at catching secret changes to data. Each block "locks in" its own content using a cryptographic hash, and every later block also stores a copy of the hash before it. So if someone tries to change old data without updating every single hash after it (which is very hard to do), the chain will notice and show that something is wrong.

To be clear, this project is not trying to copy or fully represent real systems like Bitcoin or Ethereum. Its purpose is to teach the core idea in a simple, understandable way. By keeping the project small and showing everything clearly on the screen, it becomes much easier to see, explain, and demonstrate how SHA-256 hashing, block linking (previousHash), and tamper detection work together.